

# Programmation Orientée objet Java

*Pr. Ilyass OUAZZANI TAYBI*

*E-mail : i.ouazzani@uca.ac.ma*

Département : Informatique FSSM

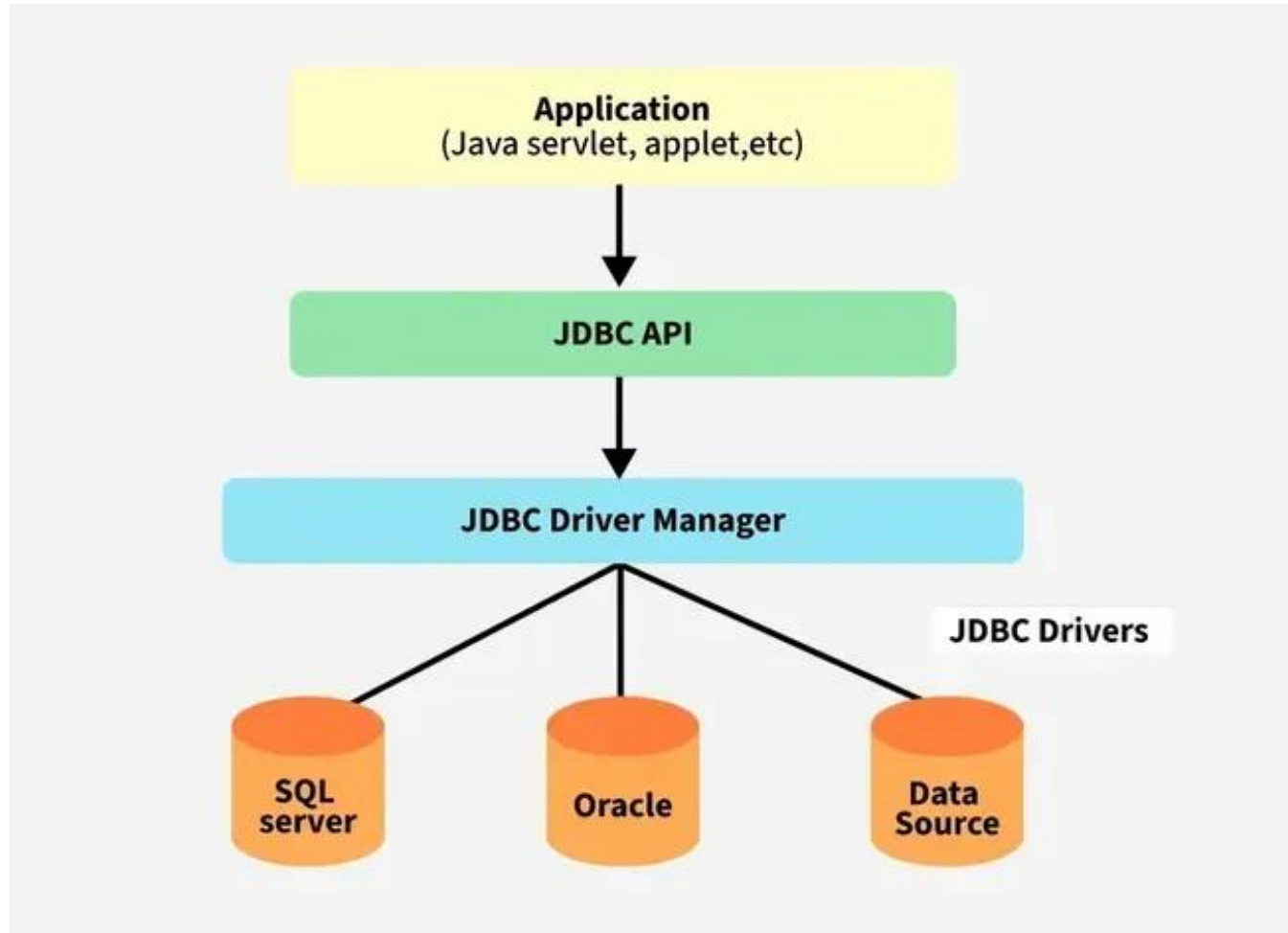
Université Cadi Ayyad

Licence SI S5

Année universitaire : 2025/2026

# Introduction à JDBC

## Vue d'ensemble



# Introduction à JDBC

## JDBC MySQL

- **JDBC (Java DataBase Connectivity)** est une **API standard** permettant d'interagir avec différentes bases de données relationnelles.
- Pour ce cours, nous allons utiliser le pilote **JDBC pour MySQL**, disponible sur le site officiel de MySQL : [www.mysql.com](http://www.mysql.com).
- Étapes pour obtenir le pilote :
  - Accédez au site de MySQL.
  - Téléchargez le fichier correspondant au connecteur JDBC : `mysql-connector-java-xxx.zip`.
- Une fois téléchargé, ajoute-le dans ton classpath.

# Introduction à JDBC

## Étapes pour utiliser JDBC

1. **Charger le pilote JDBC** : Importer et initialiser le pilote correspondant à votre base de données (ex. MySQL).
2. **Établir une connexion à la base de données** : Utiliser *DriverManager.getConnection(url, user, password)* pour se connecter.
3. **Créer une instruction (Statement)** : Préparer un objet *Statement* ou *PreparedStatement* pour exécuter vos requêtes SQL.
4. **Exécuter la requête** : Utiliser *executeQuery* pour les requêtes de type SELECT ou *executeUpdate* pour les requêtes de type INSERT, UPDATE, DELETE.
5. **Traiter les résultats** : Parcourir le *ResultSet* pour récupérer et manipuler les données retournées.
6. **Fermer la connexion** : Libérer toutes les ressources : *ResultSet*, *Statement* et *Connection*.

# Introduction à JDBC

## Étapes pour utiliser JDBC : Charger le pilote JDBC (étape 1)

Trois façons de charger un pilote JDBC :

1. *Via les propriétés du système* (avant l'appel à l'API): Définir la propriété jdbc.drivers avec le nom de la classe du pilote :

***System.setProperty("jdbc.drivers", "nomDeLaClasseDuDriver");***

2. *Au moment du démarrage de l'application* : Passer le pilote comme option à la machine virtuelle Java :

***java -Djdbc.drivers=nomDeLaClasseDuDriver MonProgramme***

3. *Pendant l'exécution du programme* : Charger la classe du pilote directement dans le code avec ***Class.forName*** :

```
try { Class.forName("nomDeLaClasseDriver");  
    } catch (ClassNotFoundException e) { e.printStackTrace(); }
```

**Remarque** : Depuis JDBC 4.0 (Java 6 et supérieur), la plupart des pilotes modernes se chargent automatiquement, donc l'appel à Class.forName n'est souvent plus nécessaire.

# Introduction à JDBC

## Étapes pour utiliser JDBC : Mettre en place une connexion JDBC (étape 2)

Pour l'ouverture d'une connexion JDBC les méthodes Java doivent spécifier :

- le nom du pilote JDBC à charger (étape 1)
- les paramètres pour localiser la base de données.
- le nom de l'utilisateur et son mot de passe

```
try { Class.forName("com.mysql.cj.jdbc.Driver"); // Charger le pilote
    String url = "jdbc:mysql://localhost/maBD";
    Connection con = DriverManager.getConnection(url, "Login", "Password");
    System.out.println("Connexion réussie !");
    con.close(); //Fermer la connexion après utilisation
} catch (ClassNotFoundException e) {
    e.printStackTrace();
} catch (SQLException e) {
    e.printStackTrace();
}
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

- L'objet JDBC **Statement** permet d'envoyer des instructions SQL à la base de données. Il est créé à partir d'une connexion JDBC active :

***Statement stmt = con.createStatement();***

- Avec cet objet, vous pouvez exécuter des requêtes SQL directement sur la base de données, comme : **SELECT** pour récupérer des données et **INSERT, UPDATE, DELETE** pour modifier des données.

```
Statement stmt = con.createStatement();  
  
ResultSet rs = stmt.executeQuery("SELECT * FROM utilisateurs");  
  
while(rs.next()){System.out.println(rs.getString("nom")+" - "+rs.getInt("age"));}  
  
stmt.close(); // Fermer le Statement après usage
```

**Remarque** : Pour des requêtes avec des paramètres, il est préférable d'utiliser **PreparedStatement**, plus sûr et flexible.

# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

- Pour des requêtes avec paramètres dynamiques, préférez *PreparedStatement* qui offre plus de sécurité et de flexibilité.

```
String sql = "SELECT * FROM utilisateurs WHERE age > ? AND ville = ?";
PreparedStatement pstmt = con.prepareStatement(sql);
//Remplacer les paramètres
pstmt.setInt(1, 18); // premier ? -> 18
pstmt.setString(2, "Marrakech"); // deuxième ? -> "Marrakech"
// Exécuter la requête
ResultSet rs = pstmt.executeQuery();
// Parcourir les résultats
while(rs.next()){System.out.println(rs.getString("nom")+" - " + rs.getInt("age")
                                   +" ans");}
pstmt.close(); // Fermer le PreparedStatement
```



# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

➤ Méthodes utiles de l'objet Statement/PreparedStatement:

| Méthode              | Rôle                                           |
|----------------------|------------------------------------------------|
| setMaxRows(int)      | Limiter le nombre de lignes du ResultSet       |
| getMaxRows()         | Récupérer la limite actuelle                   |
| setQueryTimeout(int) | Définir un délai max d'exécution (en secondes) |
| getQueryTimeout()    | Lire le délai configuré                        |

➤ Exemple :

```
Statement stmt = con.createStatement();  
stmt.setMaxRows(50); // Limite à 50 lignes max  
stmt.setQueryTimeout(5); // Timeout de 5 secondes  
ResultSet rs = stmt.executeQuery("SELECT * FROM employes");
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

Pour appeler des procédures stockées dans la base de données, JDBC fournit l'objet ***CallableStatement***. Il est créé à partir de la connexion de la même manière que ***PreparedStatement*** :

```
CallableStatement cs = con.prepareCall("{call nomProcedure(?, ?)}");
```

***CallableStatement*** permet :

- d'exécuter des procédures stockées contenant de la logique métier,
- d'envoyer des paramètres d'entrée,
- de récupérer des paramètres de sortie,
- de manipuler des valeurs de retour.

# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

Exemple d'utilisation de *CallableStatement* : Procédure stockée (MySQL)

```
CREATE PROCEDURE calculerSalaire (IN idEmp INT,  
                                IN pourcentage DOUBLE,  
                                OUT nouveauSalaire DOUBLE)  
  
BEGIN  
    SELECT salaire * (1 + pourcentage/100)  
    INTO nouveauSalaire  
    FROM employes  
    WHERE id = idEmp;  
  
END
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Créer un objet Statement (étape 3)

Exemple d'utilisation de *CallableStatement* : Code Java avec *CallableStatement*

```
// Préparer l'appel de la procedure en utilisant con
CallableStatement cs = con.prepareCall("{ call calculerSalaire(?, ?, ?) }");
//Définir les paramètres d'entrée
cs.setInt(1, 5);           // id employé
cs.setDouble(2, 10.0);     // augmentation 10%
//Définir les paramètres de sortie
cs.registerOutParameter(3, Types.DOUBLE);
//Exécuter la procédure
cs.execute(); //Récupérer le résultat (param OUT)
double salaireCalcule = cs.getDouble(3);
System.out.println("Nouveau salaire = " + salaireCalcule);
cs.close();
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Exécution des requêtes avec JDBC (étape 4)

### 1. *executeQuery()*

- Utilisée pour les requêtes SQL de type **SELECT**.
- Retourne les données dans un objet ResultSet (un tableau de résultats).
- Le ResultSet peut être vide, mais ne sera jamais null.

```
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery("SELECT * FROM utilisateurs");  
while(rs.next()) {  
    System.out.println(rs.getString("nom")+" - "+rs.getInt("age"));  
}  
stmt.close(); // Fermer le Statement après usage
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Exécution des requêtes avec JDBC (étape 4)

### 2. *executeUpdate()*

- Utilisée pour les instructions SQL de type *INSERT, UPDATE, DELETE*.
- Retourne le nombre de lignes affectées dans la base de données.
- Peut également être utilisée pour les commandes DDL (Data Definition Language) : ***CREATE TABLE, DROP TABLE*** et ***ALTER TABLE***

```
Statement stmt = con.createStatement();  
  
int lignesModifiees = stmt.executeUpdate("UPDATE departement SET  
                                     nom_departement='Informatique' WHERE id=1");  
  
System.out.println(lignesModifiees + " ligne(s) modifiée(s)");
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Exécution des requêtes avec JDBC (étape 4)

### *3. execute()*

La méthode `execute()` est utilisée lorsque l'on ne sait pas à l'avance si l'instruction SQL va :

- retourner un `ResultSet` (ex : `SELECT`)
  - ou retourner un compte de lignes modifiées (ex : `INSERT`, `UPDATE`, `DELETE`)
  - ou ne rien retourner (ex : commandes DDL : `CREATE TABLE`)
- ❖ **C'est la méthode la plus générale.**

```
boolean hasResult = stmt.execute(sql) ;
```

- Retourne `true` → si la commande produit un `ResultSet`
- Retourne `false` → si la commande produit un nombre de lignes modifiées ou rien

# Introduction à JDBC

## Étapes pour utiliser JDBC : Traiter les résultats (étape 5)

- L'objet **ResultSet** contient les résultats retournés par une requête SQL.
- Méthodes utiles (peuvent toutes lever une SQLException)
  - **close()** : Ferme le **ResultSet** et libère les ressources JDBC associées.
- À noter : le **ResultSet** est automatiquement fermé lorsque l'objet Statement associé exécute une nouvelle requête.
- **getMetaData()** : Retourne un objet **ResultSetMetaData**, qui contient des informations sur les colonnes du **ResultSet** :
  - nombre de colonnes
  - nom de chaque colonne
  - type de données
  - etc.



# Introduction à JDBC

## Étapes pour utiliser JDBC : Traiter les résultats (étape 5)

### ❖ *findColumn(String colName)*

- Cette méthode : prend le nom d'une colonne sous forme de chaîne ("nom", "age", "email", etc.) et retourne l'index numérique de cette colonne dans le ResultSet. Exemple :

```
int index = rs.findColumn("email");
```

### ❖ *getXxx(String colName)*

- Cette méthode sert à lire la valeur d'une colonne en précisant son type Java. Exemple:

```
String nom = rs.getString("nom");
```

```
int age = rs.getInt("age");
```

```
double salaire = rs.getDouble("salaire");
```

# Introduction à JDBC

## Étapes pour utiliser JDBC : Traiter les résultats (étape 5)

| Méthodes           | Descriptions                                    |
|--------------------|-------------------------------------------------|
| rs.next()          | ligne suivante                                  |
| rs.previous()      | ligne précédente                                |
| rs.absolute(i)     | aller à la ième ligne                           |
| rs.absolute(-i)    | aller à la ième ligne en partant de la dernière |
| rs.relative(i)     | descendre de i lignes                           |
| rs.relative(-i)    | remonter de i lignes                            |
| rs.afterLast()     | aller après la dernière ligne                   |
| rs.isAfterLast()   | retourne vrai si après dernière ligne           |
| rs.last()          | aller à la dernière ligne (comme absolute(-1))  |
| rs.beforeFirst()   | aller avant la première ligne                   |
| rs.isBeforeFirst() | retourne vrai si avant première ligne           |
| rs.first()         | aller à la première ligne (comme absolute(1))   |

# Introduction à JDBC

## Étapes pour utiliser JDBC : Traiter les résultats (étape 5)

### ❖ *ResultSetMetaData*

- *ResultSetMetaData* rsmd = rs.getMetaData() ;
- int *getColumnCount*() //Le nombre de colonnes
- int *getColumnDisplaySize*(int column) //La taille d'affichage d'une colonne
- String *getColumnName*(int column) //Le nom de colonne
- String *getColumnTypeName*(int column) //Le nom du type de la colonne

# Introduction à JDBC

## Étapes pour utiliser JDBC : Fermer la connexion (étape 6)

- Une fois que vous avez terminé d'utiliser JDBC, il est important de libérer les ressources en fermant les objets utilisés :

***stmt.close();***

***connection.close();***

- Pourquoi fermer ?
  - Cela libère les ressources du serveur de base de données.
  - Cela évite les fuites de connexion (connection leaks).
  - Cela permet à d'autres utilisateurs ou processus d'utiliser les connexions.

# Introduction à JDBC

## Gestion des exceptions JDBC

Presque toutes les méthodes JDBC peuvent lever une ***SQLException*** lorsqu'une erreur d'accès aux données se produit.

Lorsqu'il y a **plusieurs erreurs successives**, les **exceptions sont chaînées entre elles**.

Une ***SQLException*** fournit plusieurs informations utiles :

1. **Description de l'erreur**. Récupérée avec **getMessage()**.
2. **SQLState**. Récupérée avec **getSQLState()**.
3. **Code d'erreur fournisseur** : Récupérée avec **getErrorCode()**.
4. **Exception suivante (chaînage)**. Récupérée avec **getNextException()**.

# Introduction à JDBC

## Gestion des exceptions JDBC: Exemple d'exception SQL

```
try{
    ... // JDBC statement.
}catch (SQLException sqle) {
    while (sqle != null) {
        System.out.println("Message:      " + sqle.getMessage());
        System.out.println("SQLState:  " + sqle.getSQLState());
        System.out.println("Vendor Error: " + sqle.getErrorCode());
        sqle.printStackTrace(System.out);
        sqle = sqle.getNextException();
    }
}
```

# Programmation Orientée objet Java

*Pr. Ilyass OUAZZANI TAYBI*

*E-mail : i.ouazzani@uca.ac.ma*

Département : Informatique FSSM

Université Cadi Ayyad

Licence SI S5

Année universitaire : 2025/2026